Other Outcomes: Ordinal, Nominal, Count, Duration

Francisco Villamil

Applied Quantitative Methods II

MA in Social Sciences, Spring 2026

Logistics

- Presentations (April 16th)
  - → **5-minute presentations**, blocks of ≈5, random order, 15-min feedback after each block
    https://franvillamil.github.io/AQM2/logistics.html
  - → Send me **presentations** by email **in advance**
    3 slides max!: e.g. what, how, problems
- Rest of the course
  - → Computing session (April 23rd)
  - → Final session (April 30th), **exam** + review

Today's goals

- Understand the GLM framework as a unifying structure
- Model ordinal outcomes with ordered logit
- Model nominal (multi-category) outcomes with multinomial logit
- Model count data with Poisson and negative binomial regression
- Model time-to-event data with survival analysis and the Cox model
- Know which model to reach for and why

This is the last substantive lecture before the exam. The goal is not to make students into experts in each of these models, but to give them enough conceptual grounding and practical R syntax to apply the right tool when they encounter these outcome types in their own research. All these models share a common logic: they are generalizations of the regression framework students already know. Stress continuity with prior sessions throughout. The GLM framework is the conceptual anchor.

Roadmap

One framework, many models

Generalized Linear Models (GLMs) share three components:

Linear predictor	Link function	Distribution
$\eta_i = \mathbf{x}_i\boldsymbol{\beta}$	$g(\mu_i) = \eta_i$	$Y_i \sim F(\mu_i)$
A weighted sum of covariates	Connects mean to predictor	Data-generating process

- OLS is a GLM: identity link + normal distribution
- Logit is a GLM: logit link + Bernoulli distribution
- Choosing a model = choosing a link + distribution

Start by grounding today's models in the framework students already know. Every model they have learned — OLS, logit, probit — is a GLM. The only thing that changes across models is the link function (what transformation connects the linear predictor to the mean of the outcome) and the distributional assumption (what distribution describes the outcome). This unified perspective means that the R syntax is very similar across models (`glm()` with different `family` arguments), and the interpretation logic is analogous. Once students see this, the variety of models becomes less intimidating.

Matching outcome type to model

Outcome type	Distribution	Link	Model / Function
Continuous	Normal	Identity	OLS (`lm()`)
Binary (0/1)	Bernoulli	Logit	Logit (`glm(...,binomial)`)
Ordinal	Prop. odds	Cumul. logit	Ordered logit (`polr()`)
Nominal (>2)	Multinomial	Log-odds	Multinom. logit (`multinom()`)
Count (≥ 0)	Poisson	Log	Poisson (`glm(...,poisson)`)
Count (overdisp)	Neg. Binomial	Log	NegBin (`glm.nb()`)
Duration (time)	—	—	Cox PH (`coxph()`)

First question: what is the data-generating process for your outcome?

This table is the map for the entire lecture. Return to it at the start of each section. The key practical advice: before running any model, ask yourself what kind of thing your outcome is. Is it bounded? Can it be negative? Is it a count? Does it represent time until an event? The answers to these questions determine the appropriate model. The table is also useful for exam preparation: students should be able to map from outcome description to model family. Note that survival models (Cox) are semiparametric — they do not fit neatly into the GLM family in the standard sense, but they follow the same spirit of matching distribution to outcome type.

Roadmap

Ordinal outcomes:

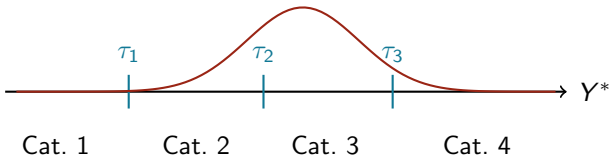
When categories have an order but not equal spacing

Ordinal outcomes are ordered categories

- Categories have a natural **ordering**, but not equal intervals
- Examples:
  - → Survey: "How satisfied are you?" (1 = not at all, 5 = very)
  - → Party ID: Strong Dem → Weak Dem → Ind → Weak Rep → Strong Rep
  - → Democracy rating: Autocracy → Hybrid → Democracy
  - → Conflict intensity: No conflict → Low → Medium → High
- The distance between "1" and "2" need not equal the distance between "2" and "3"
- OLS treats them as equally spaced — often wrong

Start with the intuition. Ordinal outcomes are very common in social science, especially in survey research (Likert scales are everywhere). The core problem with OLS is that it assumes the categories are equally spaced on the underlying scale of interest. But is the difference between "not at all satisfied" and "slightly satisfied" the same as between "satisfied" and "very satisfied"? Almost certainly not. OLS will also produce predictions that fall between categories or outside the range, which is substantively nonsensical. These problems motivate the ordered logit model.

The latent variable idea



- Y^*: unobserved continuous variable (e.g., "true" satisfaction)
- We observe category j when $\tau_{j-1} < Y^* \leq \tau_j$
- Thresholds τ_j are **estimated** from the data

The latent variable interpretation is the cleanest way to understand ordered logit. Imagine there is a true, continuous underlying attitude or propensity (e.g., genuine policy satisfaction). This variable is unobserved. What we do observe is which category the person falls into, determined by whether their latent score crosses a series of thresholds. The ordered logit model simultaneously estimates the regression coefficients (how covariates shift the latent distribution) and the threshold parameters (where the cut points are). The threshold parameters are not of primary substantive interest, but they are needed to compute predicted probabilities for each category. Connect this to the binary logit from Session 3: binary logit has a single threshold (at 0, by convention), and ordered logit generalizes this to $K - 1$ thresholds for K categories.

Ordered logit: the model

Cumulative link model (proportional odds)

$$\Pr(Y_i \leq j \mid \mathbf{x}_i) = \frac{1}{1 + \exp(-(\tau_j - \mathbf{x}_i\boldsymbol{\beta}))}$$

- One set of $\boldsymbol{\beta}$ for all thresholds (**proportional odds assumption**)
- $\beta_k > 0$ shifts Y^* upward $\rightarrow$ **higher** categories become more likely
- $\Pr(Y_i = j) = \Pr(Y_i \leq j) - \Pr(Y_i \leq j - 1)$: probability for each category by differencing cumulative probabilities

The key structural assumption of ordered logit is proportional odds: a unit change in X shifts the log-odds of being at or below any category by the same amount β. This is strong. In practice, it should be tested (Brant test in R). If violated, alternatives include the partial proportional odds model or multinomial logit (treating categories as nominal). The formula at the top models cumulative probabilities — the probability of being in category j or lower. The probability for exactly category j is obtained by differencing adjacent cumulative probabilities. This is how you produce predicted probabilities for each category, which is what students should report rather than the raw coefficients.

Ordered logit in R

```
library(MASS); library(marginaleffects)
m_ord = polr(factor(satisfaction) ~ age + educ,
     data = mydata, Hess = TRUE)
avg_slopes(m_ord) # AMEs per predictor per category
```

- `Hess = TRUE`: stores the Hessian — required for SEs
- Sign warning: raw `polr()` coefficients use a reversed sign convention — always interpret via `avg_slopes()`

Walk through the syntax. `polr()` is the workhorse for ordered logit in R. The outcome must be a factor with ordered levels — verify this before fitting. `Hess = TRUE` stores the Hessian (second-derivative matrix), which is required to compute standard errors. The raw coefficients from `polr()` are on the log-odds scale and not directly interpretable. The thresholds ("Intercepts") are also in the output. As with binary logit, the right tool for interpretation is the `marginaleffects` package: `avg_slopes()` gives AMEs for each predictor on the probability scale, separately for each response category. `predictions(m_ord)` gives the predicted probability of each category for each observation.

Ordered logit: R output

Using `housing` data from MASS: satisfaction with housing conditions

```
> m_ord = polr(Sat ~ Infl + Type + Cont, weights = Freq, data = housing, Hess = TRUE)
> summary(m_ord)

Coefficients:
               Value Std. Error t value
InflMedium     0.5664    0.10465   5.412
InflHigh       1.2888    0.12716  10.136
TypeApartment -0.5724    0.11924  -4.800
TypeAtrium    -0.3662    0.15517  -2.360
TypeTerrace   -1.0910    0.15149  -7.202
ContHigh       0.3603    0.09554   3.771

Intercepts:
            Value   Std. Error t value
Low|Medium  -0.4961  0.1248    -3.9739
Medium|High  0.6907  0.1255     5.5049

Residual Deviance: 3479.149
AIC: 3495.149
```

- **Coefficients**: log-odds scale, not directly interpretable
- **Intercepts**: estimated thresholds τ_j between categories
- No p-values — use t-values > 2 as rough guide

Walk through each section of the output. The **Coefficients** table shows the effect of each predictor on the latent variable in log-odds units. The sign tells us direction: positive means higher values of the predictor shift the distribution toward higher categories. For example, `InflHigh` (high perceived influence) has a coefficient of 1.29, meaning people who feel they have high influence over housing management are much more likely to report high satisfaction. The **Intercepts** (thresholds) define the cut points between categories on the latent scale. There are $K - 1 = 2$ thresholds for 3 categories (Low, Medium, High satisfaction). Note: `polr()` does not report p-values directly. The t-values follow an approximate normal distribution in large samples, so $|t| > 2$ is roughly significant at 5%. For formal inference, use `avg_slopes()`.

Ordered logit: marginal effects

`avg_slopes(m_ord)` — average marginal effects on probability scale

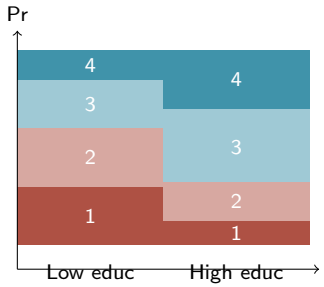
```
> avg_slopes(m_ord)

 Term  Group          Contrast Estimate Std. Error      z Pr(>|z|)
 Cont  Low    High - Low       -0.0727     0.0193  -3.77   <0.001
 Cont  Medium High - Low       -0.0064     0.0022  -2.93    0.003
 Cont  High   High - Low        0.0791     0.0206   3.83   <0.001
 Infl  Low    Medium - Low     -0.1287     0.0236  -5.46   <0.001
 Infl  Low    High - Low       -0.2619     0.0237 -11.04   <0.001
 Infl  High   Medium - Low      0.1202     0.0220   5.46   <0.001
 Infl  High   High - Low        0.2909     0.0279  10.42   <0.001
 Type  Low    Terrace - Tower   0.2239     0.0312   7.17   <0.001
 Type  High   Terrace - Tower  -0.2393     0.0316  -7.59   <0.001
```

- One row per predictor $\times$ outcome category
- **Group**: which satisfaction level (Low, Medium, High)
- E.g., high influence $\rightarrow$ +29pp probability of "High" satisfaction
- AMEs across categories **sum to zero** for each predictor

This is the key interpretive output. Each row gives the average marginal effect of a predictor on the probability of a specific outcome category. Read the `Group` column carefully: it tells you which satisfaction category the effect refers to. For example, having high perceived influence (vs. low) increases the probability of being in the "High" satisfaction group by about 29 percentage points, while decreasing the probability of being in the "Low" group by about 26 percentage points. The AMEs across all categories must sum to zero for each predictor — probability shifted out of one category must go into another. This is a good sanity check. Note: the output is trimmed here for the slide; the full output has one row per predictor per category per contrast.

Interpreting ordered logit

- Raw coefficients: log-odds — not intuitive
- **Better**: average marginal effects
  - → One AME per predictor *per category*
  - → "A one-unit increase in education changes the probability of category j by ΔP_j"
- **Best**: predicted probability plots
  - → How does $\Pr(Y = j)$ change across X?
  - → Use `plot_predictions(m_ord, condition = "educ")`



The stacked bar schematic shows the key communication tool for ordered logit: as education increases, the distribution over categories shifts toward higher categories. Each bar represents the full probability distribution over the response categories. The AMEs tell you the average slope for each category probability across the sample. Note that the AMEs across all categories must sum to zero for any given predictor — if one category's probability goes up, others must go down. This is a useful sanity check. Always plot predicted probabilities when reporting ordered logit results. A table of log-odds coefficients alone is not informative.

Application: Markov transition models

- Ordinal models can be used to study **transitions between ordered states**
  - → Democracy scores (autocracy → anocracy → democracy)
  - → Credit ratings (AAA → AA → A → . . .)
  - → Health status (healthy → mild → severe → critical)
- **Idea**: given current state S_t, model the probability of transitioning to each state at S_{t+1} using ordinal logit
- The ordinal structure constrains transitions to respect the natural ordering — large jumps (e.g., autocracy → full democracy) are possible but predicted to be rare
- Covariates shift the latent propensity, making upward or downward transitions more likely
  - → E.g., economic crisis → higher probability of moving down the scale
- In practice: **interact all covariates with S_t**

This slide shows students one application of ordinal models beyond the typical survey-item setting. A Markov transition model asks: given that a unit is in state j at time t, what is the probability it moves to each possible state at $t + 1$? If the states are ordered, an ordinal logit is a natural fit — it models the transition probabilities while respecting the ordering constraint. For example, Epstein et al. (2006) model transitions between political regime types (autocracy, partial democracy, full democracy) as an ordered outcome. The ordered logit ensures that jumping two categories is less probable than moving one step, which matches our substantive expectation. The last point is important in practice: by interacting all covariates with the current state S_t, you allow the effects to be state-dependent — e.g., the effect of an economic crisis on regime change might differ depending on whether the country is already an autocracy or a partial democracy. This is equivalent to subsetting the data by current state and fitting separate ordinal models, but the interaction approach keeps everything in one model and allows you to test whether the

Roadmap

Nominal outcomes:

Multiple unordered categories — vote choice, transport mode, conflict type

When categories have no natural order

- **Nominal outcomes**: categories, but no ranking
- Examples:
  - → Vote choice in a multi-party election (PSOE, PP, Vox, Sumar, . . .)
  - → Mode of transport (car, bus, train, bike, walk)
  - → Type of conflict (interstate, civil war, non-state)
  - → Occupation category (manager, professional, manual, service)
- Ordered logit is wrong here — it would impose an ordering
- We need a model that treats all categories symmetrically

The distinction between ordinal and nominal is important and often overlooked. If you use ordered logit on a nominally scaled outcome, you are imposing an arbitrary ordering on the categories, which will bias your results. The multinomial logit treats all categories as equally valid alternatives and estimates separate coefficients for each (relative to a reference category). Common examples in political science: vote choice models, where the alternatives are parties or candidates with no natural order. In economics: occupational choice, mode of transport. In conflict studies: type of conflict event.

Multinomial logit: the model

Softmax probabilities

$$\Pr(Y_i = j \mid \mathbf{x}_i) = \frac{\exp(\boldsymbol{\beta}_j' \mathbf{x}_i)}{\sum_{k=1}^{J} \exp(\boldsymbol{\beta}_k' \mathbf{x}_i)}$$

- J categories $\rightarrow$ $J-1$ sets of coefficients (one category is the **reference**)
- Reference category: $\boldsymbol{\beta}_1 = \mathbf{0}$ by convention
- Each β_{jk}: effect of x_k on log-odds of category j **vs. reference**
- All predicted probabilities sum to 1: $\sum_{j=1}^{J} \Pr(Y_i = j) = 1$

The softmax formula ensures all predicted probabilities are non-negative and sum to 1. This is the multinomial generalization of the logistic function. The model estimates $J-1$ separate log-odds equations, each comparing one category to the reference. If there are 5 parties, there are 4 sets of coefficients. The choice of reference category changes the coefficient values but not the predicted probabilities or the model fit. Choosing a substantively meaningful reference (e.g., the largest party, or the "status quo" option) makes interpretation easier. The coefficients tell you how covariates shift the probability of choosing category j relative to the reference.

Multinomial logit in R

```
library(nnet)

m_mnom = multinom(vote ~ age + ideology + income,
       data = beps)

# AMEs (one per predictor per category)
avg_slopes(m_mnom)
```

- First category = reference; coefficients = log-odds vs. reference
- Change reference: `relevel(factor(vote), ref = "Labour")`
- Interpret via `avg_slopes()` or `plot_predictions()`

The `nnet` package is the standard workhorse for multinomial logit in R. The `multinom()` function uses neural-network optimization under the hood, but the model is identical to standard multinomial logit. A common workflow: fit the model, then use `marginaleffects` for interpretation. The `avg_slopes()` function will return AMEs for each predictor for each category — this can be a large table, so prioritize plotting. The BEPS (British Election Panel Study) dataset in the `nnet` package is a classic teaching example for multinomial models of vote choice. Note: larger datasets require more iterations; increase with `MaxNWts` or `maxit` arguments.

Multinomial logit: R output

Using `iris` data: predicting species from flower measurements

```
> m_mnom = multinom(Species ~ Sepal.Length + Sepal.Width + Petal.Length,
+                   data = iris, trace = FALSE)
> summary(m_mnom)

Coefficients:
           (Intercept) Sepal.Length Sepal.Width Petal.Length
versicolor    15.84477    -5.106783   -9.373084     15.67647
virginica    -22.68156    -8.980237   -9.993958     28.90429

Std. Errors:
           (Intercept) Sepal.Length Sepal.Width Petal.Length
versicolor    38.86805     104.8927    164.5751     79.52894
virginica     39.08951     104.9167    164.5911     79.55610

Residual Deviance: 23.77288
AIC: 39.77288
```

- One row of coefficients **per non-reference category**
- Reference = *setosa*; each row = log-odds vs. reference
- Coefficients are log-odds → hard to interpret directly
- Use `avg_slopes()` or `plot_predictions()` instead

Walk through the output structure. The key difference from binary logit: there is one row of coefficients for each non-reference category. Here the reference category is *setosa* (the first level), so we get one row for *versicolor* (log-odds of versicolor vs. setosa) and one for *virginica* (log-odds of virginica vs. setosa). Each coefficient gives the change in log-odds of that category relative to the reference for a one-unit change in the predictor. For example, Petal.Length has a coefficient of 15.7 for versicolor and 28.9 for virginica — longer petals strongly increase the odds of both categories relative to setosa, but more so for virginica. The large standard errors in this example reflect near-perfect separation. The raw coefficients are rarely reported; always interpret via marginal effects or predicted probabilities.

Multinomial logit: marginal effects

`avg_slopes(m_mnom)` — average marginal effects on probability scale

```
> avg_slopes(m_mnom)

         Term      Group    Estimate Std. Error       z Pr(>|z|)
 Petal.Length setosa     -0.00003018   0.001756  -0.017   0.9863
 Petal.Length versicolor -0.31811938   0.023102 -13.770   <0.001
 Petal.Length virginica   0.31814956   0.023035  13.812   <0.001
 Sepal.Length setosa      0.00000983   0.000578   0.017   0.9864
 Sepal.Length versicolor  0.09315278   0.032003   2.911   0.0036
 Sepal.Length virginica  -0.09316261   0.031998  -2.912   0.0036
 Sepal.Width  setosa      0.00001805   0.001106   0.016   0.9870
 Sepal.Width  versicolor  0.01491495   0.054905   0.272   0.7859
 Sepal.Width  virginica  -0.01493299   0.054891  -0.272   0.7856
```

- One row per predictor × category (like ordered logit)
- 1cm longer petal → −32pp *versicolor*, +32pp *virginica*
- AMEs across categories **sum to zero** for each predictor

The marginal effects table is the most useful output for interpretation. Each row tells you: for a one-unit increase in the predictor, how much does the probability of each species change on average? Petal length is the key discriminator: one additional centimeter reduces the probability of versicolor by 32 percentage points and increases the probability of virginica by the same amount, with essentially no effect on setosa (which has very short petals). Sepal width is not significant. As with ordered logit, the AMEs across all categories sum to zero for each predictor — probability is redistributed across categories, not created or destroyed. This is the output to report in a paper or presentation, not the raw log-odds coefficients.

The IIA assumption

Independence of Irrelevant Alternatives (IIA): odds ratio btw any two alternatives **unaffected** by presence of other alternatives

$$\frac{\Pr(Y=j)}{\Pr(Y=k)} = \exp\big((\boldsymbol{\beta}_j - \boldsymbol{\beta}_k)'\mathbf{x}\big)$$

- The famous **red bus / blue bus** problem:
  - → 50% car, 50% red bus — now introduce a blue bus
  - → IIA implies: 33% car, 33% red bus, 33% blue bus
  - → Reality: 50% car, 25% red bus, 25% blue bus
- IIA holds when alternatives are **genuinely distinct**
- Violations: similar alternatives that substitute for each other

The IIA assumption is the key limitation of multinomial logit. It works well when alternatives are substantively distinct and do not substitute closely for each other. It breaks down when alternatives are similar: adding a blue bus (identical to the red bus) to a car/red-bus choice set should not pull equally from both alternatives. In political science, IIA is often violated when there are ideologically similar parties (e.g., two center-left parties). If IIA is violated, alternatives include nested logit (which groups similar alternatives) or mixed logit models. The Hausman-McFadden test can be used to test IIA formally, though it has well-known power problems. The practical advice: think carefully about whether your alternatives are genuinely distinct before using multinomial logit.

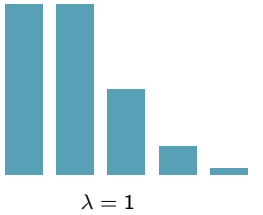
Roadmap

Count data:

Non-negative integers — conflicts, protests, bills, articles

Count outcomes

- Outcomes that are non-negative integers: 0, 1, 2, 3, ...
- Examples: conflict events, bills, protests, publications, attacks
- **Why not OLS?**
  - → Counts ≥ 0: OLS can predict negative values
  - → Variance increases with the mean
  - → Strongly right-skewed; often many zeros



$\lambda = 1$

$\lambda = 4$

Count data are ubiquitous in political science, sociology, and economics. The problem with OLS is exactly the same kind of mismatch we saw with binary outcomes: OLS is designed for continuous, symmetric, unbounded outcomes. Count data are bounded below at zero, typically right-skewed, and discrete. The heteroskedasticity problem is especially acute: for low expected counts the variance is small, but for high expected counts the variance is large. OLS assumes constant variance. The natural distribution for counts is the Poisson, which has the elegant property that its mean equals its variance.

The Poisson model

Poisson regression

$$Y_i \sim \text{Poisson}(\mu_i), \quad \mu_i = \exp(\mathbf{x}_i\boldsymbol{\beta})$$

$$\Leftrightarrow \quad \log(\mu_i) = \mathbf{x}_i\boldsymbol{\beta}$$

- $\mu_i = E[Y_i \mid \mathbf{x}_i]$: expected count (always > 0)
- The **log link** ensures predictions are always non-negative
- Coefficient interpretation: $\exp(\beta_k)$ = **multiplicative** change in expected count
  - → $\beta_k = 0.3 \Rightarrow \exp(0.3) \approx 1.35$: 35% increase in expected count
- Poisson assumption: $\text{Var}(Y_i) = E[Y_i]$ (mean = variance)

Walk through the Poisson model carefully. The log link is what ensures non-negative predictions: exponentiating a linear predictor always gives a positive number. The interpretation of coefficients as multiplicative effects is a key difference from OLS. In OLS, β is additive: a one-unit increase in X adds β to the outcome. In Poisson, $\exp(\beta)$ is multiplicative: a one-unit increase in X multiplies the expected count by $\exp(\beta)$. The Poisson distribution has a single parameter μ that equals both the mean and the variance. This is the "equidispersion" assumption. It is often violated in practice — real count data are almost always overdispersed (variance exceeds mean). This leads to the negative binomial model.

Poisson in R and interpretation

```
# Fit Poisson model
m_pois = glm(n_conflicts ~ gdp_pc + pop_l,
     family = "poisson", data = mydata)

summary(m_pois)
exp(coef(m_pois)) # incidence rate ratios
```

Raw coefficient	**Exponentiated**
$\hat{\beta}_{\texttt{gdp\_pc}} = -0.24$	$\exp(-0.24) \approx 0.79$
Hard to interpret directly	$\rightarrow$ 21% fewer conflicts per unit GDP

Also: `avg_slopes(m_pois)` gives AMEs on count scale

The two-column layout illustrates the standard reporting practice: always exponentiate Poisson coefficients. The exponentiated coefficient is called an "incidence rate ratio" (IRR): it is the ratio of expected counts for a one-unit change in X. An IRR of 0.79 means 21% fewer expected events; an IRR of 1.35 means 35% more. Note that `avg_slopes()` from `marginaleffects` gives marginal effects on the count scale (additive), not on the log scale. Both are useful: IRRs for proportional comparisons, AMEs for absolute changes. For models with offsets (e.g., controlling for population exposure), add `offset(log(population))` to the formula.

Poisson: R output

Using `warpbreaks`: number of breaks in yarn by wool type and tension

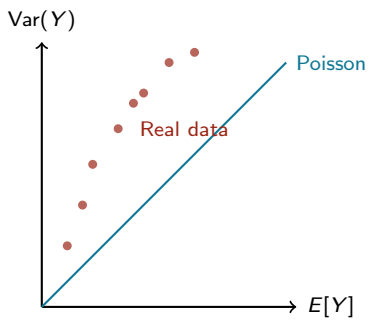
```
> m_pois = glm(breaks ~ wool + tension, family = "poisson", data = warpbreaks)
> summary(m_pois)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  3.69196    0.04541  81.302  < 2e-16 ***
woolB       -0.20599    0.05157  -3.994 6.49e-05 ***
tensionM    -0.32132    0.06027  -5.332 9.73e-08 ***
tensionH    -0.51849    0.06396  -8.107 5.21e-16 ***

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 297.37  on 53  degrees of freedom
Residual deviance: 210.39  on 50  degrees of freedom
AIC: 493.06

> exp(coef(m_pois))
(Intercept)       woolB    tensionM    tensionH
     40.124       0.814       0.725       0.595
```

- Coefficients on **log scale** $\rightarrow$ exponentiate for IRRs
- `woolB`: $\exp(-0.21) = 0.81 \rightarrow$ 19% fewer breaks
- Red flag: residual deviance (210) $\gg$ df (50) $\rightarrow$ overdispersion!

Walk through the output top to bottom. The coefficients table looks like any GLM output: estimates, standard errors, z-values, p-values. The key interpretive step is exponentiation. `woolB` = `-0.206`: $\exp(-0.206) = 0.814$, meaning wool type B has about 19% fewer breaks than wool type A (the reference). `tensionH` = `-0.519`: $\exp(-0.519) = 0.595$, meaning high tension reduces breaks by about 40% compared to low tension. The line "Dispersion parameter for poisson family taken to be 1" is important: Poisson *assumes* dispersion equals 1. But look at the deviance: residual deviance is 210 on 50 df — a ratio of 4.2, far above 1. This is a clear sign of overdispersion. The standard errors are likely too small, and we should consider a negative binomial model instead.

The overdispersion problem

- Poisson requires: $\text{Var}(Y) = E[Y]$
- Real data almost always: $\text{Var}(Y) \gg E[Y]$
- Consequences of ignoring overdispersion:
  - → Standard errors too **small**
  - → p-values too **small**
  - → False positives
- Quick check:
  Residual deviance $\gg$ df



Overdispersion is the single most common problem with Poisson regression in practice. Almost all real count data have variance greater than the mean, because counts reflect heterogeneous processes (some observations have intrinsically high rates, others low, regardless of the covariates). When you ignore overdispersion, the model is overconfident: it reports standard errors that are too small, leading to inflated t-statistics and misleadingly small p-values. The quick diagnostic: look at the ratio of residual deviance to residual degrees of freedom in the `summary()` output. If this ratio is much greater than 1 (e.g., > 1.5 or 2), overdispersion is a concern. The formal test is `AER::dispersiontest(m_pois)`.

Negative binomial regression

Negative Binomial model

$$Y_i \sim \text{NegBin}(\mu_i, \theta), \quad \log(\mu_i) = \mathbf{x}_i\boldsymbol{\beta}$$

$$\text{Var}(Y_i) = \mu_i + \frac{\mu_i^2}{\theta}$$

- $\theta > 0$: dispersion parameter estimated from the data
  - → $\theta \to \infty$: approaches Poisson (no overdispersion)
  - → Small θ: high overdispersion
- Same interpretation as Poisson: $\exp(\beta_k)$ = incidence rate ratio
- In R: `MASS::glm.nb(y ~ x1 + x2)`

The negative binomial distribution adds a dispersion parameter θ (sometimes written as $\alpha = 1/\theta$) that allows the variance to exceed the mean. Conceptually, the negative binomial mixes Poisson distributions: each unit i has its own Poisson rate μ_i, but these rates themselves follow a gamma distribution across the population. The result is a distribution with heavier tails than Poisson and a variance that scales with μ^2/θ in addition to μ. In practice, the negative binomial is almost always preferable to Poisson for social science count data, because overdispersion is the norm rather than the exception. The syntax is nearly identical to Poisson, which makes switching easy. The coefficient interpretation (via $\exp(\beta)$) is the same.

Negative binomial: R output

Same `warpbreaks` data — compare with Poisson output

```
> m_nb = glm.nb(breaks ~ wool + tension, data = warpbreaks)
> summary(m_nb)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)   3.6734     0.0979  37.520  < 2e-16 ***
woolB        -0.1862     0.1010  -1.844   0.0651 .
tensionM     -0.2992     0.1217  -2.458   0.0140 *
tensionH     -0.5114     0.1237  -4.133 3.58e-05 ***

(Dispersion parameter for Negative Binomial(9.9444) family taken to be 1)

    Null deviance: 75.464  on 53  degrees of freedom
Residual deviance: 53.723  on 50  degrees of freedom
AIC: 408.76
              Theta:  9.94
          Std. Err.:  2.56

> exp(coef(m_nb))
(Intercept)       woolB    tensionM    tensionH
     39.384       0.830       0.741       0.600
```

- **SEs are larger** than Poisson (e.g., `woolB`: 0.10 vs. 0.05)
- `woolB` now only marginal ($p = 0.065$) — was "highly significant"
- Deviance/df ≈ 1.07 — overdispersion is handled

This is the key comparison slide. Use the same data as the Poisson model so students can see the differences directly. Three things to highlight: (1) The coefficient *estimates* are similar (NegBin: -0.186 vs. Poisson: -0.206 for `woolB`), but (2) the *standard errors* are roughly doubled (0.10 vs. 0.05). This is because the negative binomial correctly accounts for the extra-Poisson variation. As a result, `woolB` goes from highly significant ($p < 0.001$) to only marginally significant ($p = 0.065$). The Poisson model was overconfident. (3) The residual deviance is now 53.7 on 50 df (ratio ≈ 1.07), indicating the model fits well. The Theta parameter ($\hat{\theta} = 9.94$) is the estimated dispersion — not huge but clearly needed. The AIC is also much lower (409 vs. 493), confirming the NegBin fits better.

Count models: a summary

Model	R function	When to use
Poisson	`glm(...,poisson)`	Equidispersed counts
Negative binomial	`MASS::glm.nb()`	Overdispersed counts
Zero-inflated Poisson	`pscl::zeroinfl()`	Excess zeros, equidisp.
Zero-inflated NegBin	`pscl::zeroinfl()`	Excess zeros, overdisp.

- **Default**: start Poisson, test overdispersion, switch to NegBin if needed
- Zero-inflation: two-process model (structural vs. count zeros)
- All share the log link: $\exp(\hat{\beta})$ = incidence rate ratio (IRR)

This summary slide gives students a practical decision tree for count models. The default recommendation: always start with Poisson, inspect the residual deviance/df ratio and run `AER::dispersiontest()`, and switch to negative binomial if overdispersion is present. Zero-inflated models (`pscl::zeroinfl()`) are for situations where there are more zeros than any count distribution can explain — e.g., a peace researcher studying conflict events, where many country-years have zero events structurally (the country is at peace) and a separate process generates events for those at risk. The zero-inflated model has two parts: a logistic regression for structural zeros and a count model for the rest. Mention this but do not go deep — it is a more advanced topic.

Zero-inflated models

- Sometimes data have **more zeros than any count distribution predicts**
  - → Conflict events: many country-years at peace (structural zeros)
  - → Patent counts: many firms never innovate
- **Two-process model**:
  - → Process 1 (logit): is the unit "at risk" of a positive count?
  - → Process 2 (Poisson/NegBin): if at risk, how many events?
- Distinguishes **structural zeros** (unit can't have the event) from **sampling zeros** (could happen, just didn't)
- In R: `pscl::zeroinfl(y ~ x | z, dist = "negbin")`
  - → `x`: predictors for the count process
  - → `z`: predictors for the zero-inflation process

This slide introduces the intuition behind zero-inflated models without going into the math. The key idea is that sometimes zeros come from two different processes: some units are structurally unable to produce events (e.g., a stable democracy will not have a civil war onset), while others are at risk but happen to have zero events in the observation period. A standard Poisson or negative binomial can only model one type of zero. The zero-inflated model separates these with a logistic regression that predicts structural zeros and a count model for the at-risk population. The predictors in the two parts can differ — for example, regime type might predict structural peace (zero-inflation part) while GDP and population predict the intensity of violence (count part). In practice, if you see a histogram with a huge spike at zero and a long right tail, consider a zero-inflated model. Compare with AIC or Vuong test (`pscl::vuong()`).

Roadmap

Duration analysis:

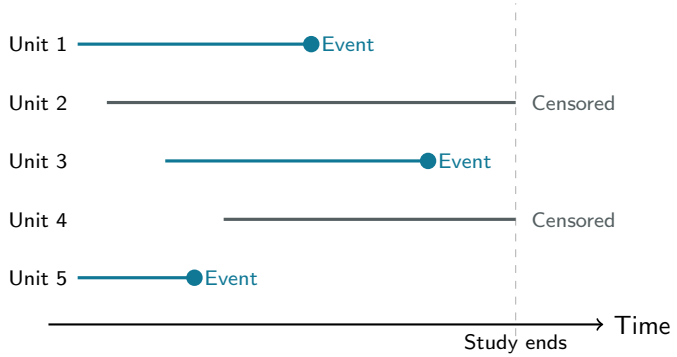
Time until an event — war onset, cabinet survival, regime collapse

Time-to-event outcomes

- Outcomes that record **how long until something happens**
- Examples:
  - → Time until a war begins (from peace)
  - → Duration of a coalition government in office
  - → Time to re-arrest after prison release
  - → How long an authoritarian regime survives
  - → Time until a bill is passed (from introduction)
- Why not OLS on duration?
  - → Durations are ≥ 0 and right-skewed
  - → **Censoring**: many observations end without the event occurring

Duration analysis (also called survival analysis, event history analysis, or failure time analysis) is the appropriate framework when the outcome is a time until an event. It is very widely used in political science (regime duration, conflict duration, cabinet survival), epidemiology (time until death, disease relapse), sociology (unemployment duration, marriage dissolution), and engineering (time until equipment failure). The two concepts students need to understand are censoring and the hazard function. Censoring is the key practical challenge: many spells end not because the event occurred, but because the observation window closed. Ignoring censoring and running OLS on the observed durations would give biased estimates.

The censoring problem



Censored units: event did not occur during observation window

This diagram shows the classic representation of survival data. Each horizontal line is a spell (the time a unit is under observation). Spells ending with a filled circle experienced the event; spells reaching the study's end without the event are right-censored. The key point: we must not throw away the censored observations. They contain information — we know the event did not occur at least until the censoring time. Dropping censored units would oversample short spells (the ones that ended with events), biasing duration estimates downward. Survival models handle censoring properly by including censored observations in the risk set until they are censored, but not expecting an event from them. In R, censoring is encoded in the `Surv(time, event)` object.

Discussion

A researcher studies how long democracies survive.

She drops all country-spells that are still ongoing at the end of the dataset, keeping only countries where a democratic breakdown was observed.

What is wrong with this decision?

This is the classic censoring bias problem. If the researcher drops right-censored observations (countries still democratic at the end of the observation window), she is systematically removing long-surviving democracies from the sample. The remaining data will only contain breakdowns, which will make the estimated hazard much higher than it actually is. In other words, she will overestimate the risk of democratic breakdown. Survival models handle this correctly: censored observations stay in the risk set until they are censored, contributing information that the event had not yet occurred. Dropping censored observations is one of the most common mistakes in applied survival analysis.

Key concepts: survival and hazard

Survival function $S(t)$

$S(t) = \Pr(T > t)$

Probability of surviving past time t

$S(0) = 1$, $S(\infty) = 0$

Monotonically non-increasing

Hazard function $h(t)$

$$h(t) = \frac{f(t)}{S(t)}$$

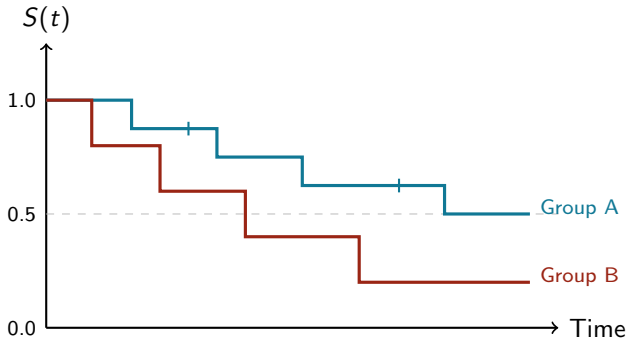
Instantaneous rate of event at t, conditional on survival to t

"Danger per unit time at time t"

- High $h(t)$: event is likely soon
- $S(t)$ and $h(t)$ are two views of the same object
- Covariates shift $h(t)$ — that is what regression models

The survival function and hazard function are the two fundamental objects in duration analysis. The survival function $S(t)$ gives the probability that the event has not yet occurred by time t. It starts at 1 (everyone is at risk) and decreases toward 0. The hazard function $h(t)$ is the instantaneous rate at which events occur among those still "at risk" at time t. The relationship $h(t) = f(t)/S(t)$ connects the hazard to the density $f(t)$ and survival $S(t)$ of the event time distribution. The hazard is what survival regression models: covariates are hypothesized to multiplicatively shift the hazard. Intuitively: a regime with a high hazard at time t is likely to fall soon; one with a low hazard is stable.

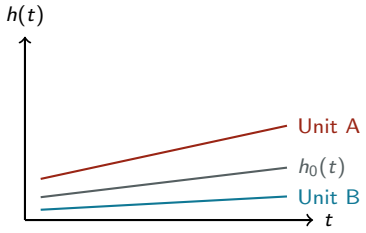
Kaplan-Meier: visualizing survival



Kaplan-Meier estimator: non-parametric, no covariates, visual comparison of groups

The Kaplan-Meier (KM) estimator is the standard first step in survival analysis. It is non-parametric: it makes no distributional assumptions and does not include covariates. It simply estimates the survival probability at each event time by updating the fraction of the risk set that experiences the event. The KM curve is always a step function, dropping at each observed event time. Censored observations are marked with tick marks on the curve. The KM curve is a powerful visual tool: you can immediately see whether Group A has higher survival than Group B, whether survival differences emerge early or late, and how many observations remain in the risk set over time. In R: `survfit(Surv(time, event) ~ group, data = mydata)` and then `survminer::ggsurvplot()`.

The proportional hazards idea



- **Key idea**: each unit has its own hazard, but all are proportional shifts of the same baseline $h_0(t)$
- The ratio $h_i(t)/h_j(t)$ is **constant over time** — curves never cross
- Violation: if the gap between lines closes or reverses over time ⇒ PH assumption fails

This diagram makes the proportional hazards (PH) assumption visually intuitive before the algebra. All hazard curves are multiples of the same baseline function $h_0(t)$ – – – *theyhavethesameshapebutdifferentlevels.Thismeansthehazardratiobetweenanytwo* *plotlog*$(-log(S(t)))$*fortwogroups* – – – *underPHtheseshouldbeparallellines.Violationsoccurwhentheeffectofacovariatechar* *interactiontermorstratifybytheoffendingcovariate*.

Cox proportional hazards model

Cox PH model

$$h_i(t) = h_0(t) \cdot \exp(\mathbf{x}_i\boldsymbol{\beta})$$

- $h_0(t)$: **baseline hazard** — left completely unspecified (semiparametric)
- $\exp(\mathbf{x}_i\boldsymbol{\beta})$: multiplicative shift of the baseline
- **Proportional hazards assumption**: the hazard ratio between any two units is *constant over time*
- $\exp(\beta_k)$: **hazard ratio** for a one-unit increase in x_k
  - → $\exp(\beta_k) > 1$: higher hazard (event happens sooner)
  - → $\exp(\beta_k) < 1$: lower hazard (event happens later)

The Cox model is the workhorse of survival regression in social science. Its key advantage is semiparametric: it does not require specifying the baseline hazard $h_0(t)$, which is often unknown and of no substantive interest. The model only specifies how covariates shift the hazard multiplicatively. The proportional hazards (PH) assumption means that the ratio of hazards between any two units remains constant over time — only the level changes. This is testable: a plot of $\log(-\log S(t))$ for two groups should show parallel curves under PH. Violations of PH suggest that the effect of a covariate changes over time, which requires either stratification or time-varying coefficients. The hazard ratio interpretation is analogous to the incidence rate ratio in Poisson regression: multiplicative effect on the event rate.

Cox model in R

```
library(survival); library(survminer)

# Kaplan-Meier estimator
km_fit = survfit(Surv(time, event) ~ group, data = d)
ggsurvplot(km_fit, conf.int = TRUE)

# Cox PH model
cox_fit = coxph(Surv(time, event) ~ x1 + x2, data = d)
summary(cox_fit) # shows hazard ratios
exp(coef(cox_fit))
```


Walk through the syntax. The `Surv(time, event)` object is the key data structure: `time` is the duration variable, `event` is a binary indicator for whether the event occurred (1) or the observation was censored (0). This object is always the left-hand side of the formula. `survfit()` fits the Kaplan-Meier estimator; `ggsurvplot()` from the `survminer` package creates publication-quality KM plots with confidence bands and risk tables. `coxph()` fits the Cox proportional hazards model. The output includes coefficients on the log-hazard scale and, when you run `summary()`, the exponentiated hazard ratios. Always test the PH assumption with `cox.zph(cox_fit)`: a significant test indicates time-varying effects that violate the PH assumption.

Cox PH: R output

Using `lung` data (survival pkg): survival of lung cancer patients

```
> cox_fit = coxph(Surv(time, status) ~ age + sex + ph.ecog, data = lung)
> summary(cox_fit)

  n= 227, number of events= 164

             coef exp(coef) se(coef)      z Pr(>|z|)
age      0.011067  1.011128 0.009267  1.194  0.23242
sex     -0.552612  0.575445 0.167739 -3.294  0.00099 ***
ph.ecog  0.463728  1.589991 0.113577  4.083 4.45e-05 ***

         exp(coef) exp(-coef) lower .95 upper .95
age         1.0111     0.9890    0.9929    1.0297
sex         0.5754     1.7378    0.4142    0.7994
ph.ecog     1.5900     0.6289    1.2727    1.9864

Concordance= 0.637  (se = 0.025 )
Likelihood ratio test= 30.5  on 3 df,   p=1e-06
Wald test            = 29.93  on 3 df,   p=1e-06
Score (logrank) test = 30.5  on 3 df,   p=1e-06
```

- `exp(coef)` = **hazard ratios** — the key quantity to report
- `sex` (2 = female): HR = 0.58 → 42% *lower* hazard (longer survival)
- `ph.ecog`: HR = 1.59 → 59% *higher* hazard per unit increase
- Concordance = 0.64: moderate predictive accuracy

Walk through the output carefully. The summary shows two tables: (1) the coefficients on the log-hazard scale with standard errors and p-values, and (2) the exponentiated coefficients (hazard ratios) with 95% confidence intervals. The hazard ratio is the key quantity. `sex`: the `lung` dataset codes sex as 1 = male, 2 = female. The hazard ratio of 0.575 means females have about 42% lower hazard of death — they survive longer. `ph.ecog` (ECOG performance score, 0 = fully active to 4 = bedridden): each unit increase in performance score (i.e., worse health) increases the hazard by 59%. Age is not significant. The Concordance statistic (0.637) measures predictive discrimination — analogous to AUC for binary outcomes. The three global tests at the bottom test whether the model as a whole is significant. All three agree ($p < 0.001$). Note the line "n = 227, number of events = 164" — 164 deaths were observed, 63 observations were censored (still alive at end of study).

Interpreting the Cox model

	Coef	Exp(coef)	SE	p-value
`gdp_pc`	-0.31	0.73	0.08	< 0.001
`military`	0.64	1.90	0.15	0.001
`log_pop`	0.18	1.20	0.10	0.07

- $\exp(\hat{\beta}_{\texttt{gdp\_pc}}) = 0.73$: richer countries have 27% **lower** hazard of event
- $\exp(\hat{\beta}_{\texttt{military}}) = 1.90$: military regimes face 90% **higher** hazard
- Hazard ratio < 1: covariate *reduces* event rate $\rightarrow$ *increases* survival
- Hazard ratio > 1: covariate *increases* event rate $\rightarrow$ *decreases* survival

Walk through the output table carefully. The "Coef" column gives log-hazard coefficients, which are hard to interpret. The "Exp(coef)" column gives hazard ratios, which are the quantities to report. The direction of the hazard ratio can be counterintuitive: a hazard ratio less than 1 means lower hazard, which means longer survival. GDP per capita coefficient: richer countries have lower hazard of regime collapse (or war, or whatever the event is), i.e., they are more stable. Military regime coefficient: military regimes face nearly twice the hazard compared to non-military regimes. These are interpreted as effects conditional on survival to time t (the risk set at each moment). Note: the PH assumption must be checked — `cox.zph(cox_fit)` tests whether the coefficients are constant over time.

Roadmap

Pulling it together:

Which model for which data?

Choosing the right model

If your outcome is...	**Use**	**Key check**
Continuous	OLS	Linearity, homoskedasticity
Binary (0/1)	Logit / LPM	Rare events → prefer logit
Ordered categories	Ordered logit	Test PO assumption (Brant)
Unordered categories	Multinomial logit	IIA plausible?
Non-negative integer	Poisson	Test overdispersion
Overdispersed count	Neg. Binomial	Often the safe default
Many zeros	Zero-inflated	Two-process data?
Time until event	Cox PH	Test PH assumption

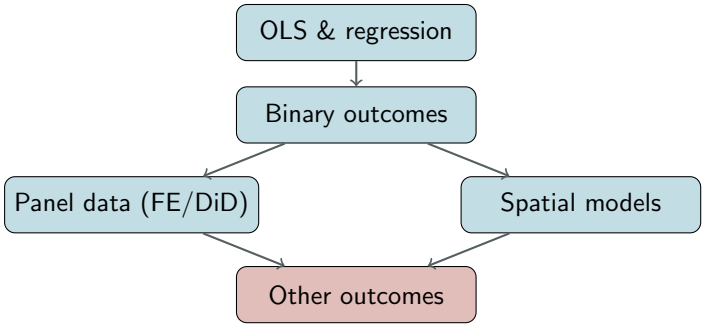
This table is a condensed version of the map from the opening section, now with diagnostic checks added. Encourage students to memorize the first two columns and think of the third column as a reminder to always validate the model's assumptions. The key message: the choice of model is driven by the data-generating process, not by habit or convention. Ask: what kind of thing is my outcome? That is the primary question. Then ask: does the model I chose make sense given the structure of my data? The diagnostic checks are not optional extras — they are part of good applied practice. For the exam, students should be able to identify the appropriate model from an outcome description and justify their choice.

Practical checklist

- **Step 1**: What is the DGP? Identify the outcome type
- **Step 2**: Choose model family accordingly (see table)
- **Step 3**: Estimate the model in R
- **Step 4**: Check assumptions
  - → Counts: overdispersion test → switch to NegBin if needed
  - → Ordinal: Brant test for proportional odds
  - → Nominal: IIA plausibility (substantive reasoning)
  - → Duration: Schoenfeld residuals for PH assumption
- **Steps 5–6**: Interpret via AMEs / predicted probabilities (**never raw coefficients alone**); visualize results

This is the practical workflow students should internalize. Stress Step 5 especially: raw coefficients in all the models we have covered today (ordinal, multinomial, count, survival) are on transformed scales (log-odds, log-count, log-hazard) that are not directly interpretable. Students must always convert to a meaningful scale before reporting. The `marginaleffects` package (`avg_slopes()`, `predictions()`, `plot_predictions()`) does this for all model types. Also stress Step 6: a plot communicates more effectively than any table of numbers, especially for models with multiple response categories (ordinal, multinomial) or continuous predictors.

This semester



Today: ordinal, nominal, count, duration

(Computing session on April 23rd)

Close with a map of the course. Students started with OLS (Sessions 1–2), moved to binary outcomes and interpretation (Sessions 3–4), then panel data with fixed effects and DiD (Sessions 5–6), then spatial models (Sessions 7–8), and now today cover other outcome types. This gives students a sense of the breadth of the toolkit they have acquired. Each extension follows the same logic: identify a feature of the data that OLS mishandles, choose a model that accommodates that feature. The unifying thread is the GLM framework introduced at the start of today's session. Emphasize that the skills they have developed — thinking about DGPs, checking assumptions, interpreting via marginal effects, visualizing results — apply across all these model families.

For the exam

- Be able to **identify** the right model given an outcome description
- Know the key **assumptions** and how to check them:
  - → Proportional odds (ordered logit)
  - → IIA (multinomial logit)
  - → Equidispersion (Poisson)
  - → Proportional hazards (Cox)
- Interpret coefficients correctly:
  - → Ordered/multinomial logit: AMEs or predicted probabilities
  - → Poisson/NegBin: $\exp(\hat{\beta})$ = incidence rate ratio
  - → Cox PH: $\exp(\hat{\beta})$ = hazard ratio
- Know the relevant **R functions**

Practical exam preparation slide. Encourage students to make a reference card with the model family, R function, link function, coefficient interpretation, and key assumption for each model type. The interpretive machinery is always the same: find the transformed scale (log-odds, log-count, log-hazard), exponentiate to get the multiplicative effect, or use `marginaleffects` to get additive AMEs. The R functions are: `polr()` for ordered logit, `multinom()` for multinomial logit, `glm(..., poisson)` or `MASS::glm.nb()` for counts, `coxph()` for survival. These are the building blocks students need for their own research.

Questions?